

AGENTS.md

1. Scopo del file

Questo file definisce le regole obbligatorie per qualunque sviluppatore o agente AI che lavori sul progetto Manual Backup.

Il riferimento funzionale principale è:

CONTRACT.md

In caso di conflitto tra implementazione e contratto, prevale `CONTRACT.md`.

Non modificare il comportamento del prodotto per interpretazione personale.

2. Regola fondamentale

Il progetto deve restare una piccola applicazione VB.NET Windows Forms per backup manuale di file tramite Robocopy.

Non trasformare il progetto in:

- servizio Windows;
 - applicazione residente;
 - scheduler;
 - sincronizzatore automatico;
 - clone disco;
 - sistema di imaging;
 - backup cloud;
 - motore proprietario di copia.
-

3. Prima di modificare il codice

Prima di applicare qualunque modifica:

1. leggere `CONTRACT.md`;
2. leggere questo file;
3. verificare la struttura attuale del progetto;
4. individuare classi, controlli e funzioni già esistenti;
5. riutilizzare il codice esistente quando equivalente;
6. evitare nuove astrazioni non necessarie;
7. verificare che la modifica non alteri comportamenti osservabili;
8. indicare eventuali punti non coperti dal contratto.

Non introdurre modifiche architetturali senza approvazione esplicita.

4. Divieti di sicurezza

Non aggiungere mai automaticamente i seguenti parametri Robocopy:

```
/MIR  
/PURGE  
/MOVE  
/MOV  
/IS  
/IT
```

Non aggiungere codice che:

- elimina file dalla destinazione;
- elimina file dalla sorgente;
- sposta file dalla sorgente;
- formatta dischi;
- modifica partizioni;
- modifica il registro di Windows;
- installa servizi;
- crea attività pianificate;
- acquisisce proprietà di file;
- cambia permessi;
- disattiva protezioni Windows;
- esegue comandi PowerShell non necessari;
- esegue comandi shell arbitrari provenienti dal JSON;
- consente l'iniezione di argomenti nella riga di comando.

Ogni funzione distruttiva richiede approvazione esplicita prima dell'implementazione.

5. Contratti da non modificare

Non modificare senza autorizzazione:

- nomi dei campi JSON;
- struttura del JSON;
- semantica dei campi;
- modalità `AppDataMode` ;
- nomi dei controlli UI già concordati;
- comportamento degli exit code Robocopy;
- esclusioni hardcoded;
- regola `ExitCode < 8 = completato` ;
- assenza di cancellazioni;
- esecuzione esclusivamente manuale;
- motore Robocopy;

- piattaforma Windows 11;
- linguaggio VB.NET;
- framework Windows Forms.

Se una modifica sembra necessaria, proporla prima indicando:

- problema;
- modifica proposta;
- impatto;
- compatibilità;
- rischi;
- alternativa conservativa.

6. Configurazione JSON

La configurazione iniziale deve mantenere questa forma:

```
{
  "SourceRoot": "C:\\Users\\Francesco",
  "DestinationRoot": "D:\\Backup\\Francesco",
  "IncludedFolders": [
    "Desktop",
    "Documents",
    "Downloads",
    "Pictures",
    "Videos",
    "Music"
  ],
  "ExcludedFolders": [
    "AppData\\Local\\Temp",
    "AppData\\Local\\Microsoft\\Windows\\INetCache"
  ],
  "ExcludedFiles": [
    "*.tmp",
    "*.temp",
    "thumbs.db",
    "desktop.ini"
  ],
  "AppDataMode": "Excluded",
  "IncludedAppDataFolders": [],
  "LogFolder": "D:\\Backup\\Logs"
}
```

Non rinominare proprietà.

Non aggiungere proprietà senza approvazione.

Non rimuovere proprietà.

Non cambiare tipo ai valori.

La configurazione deve essere deserializzata con modelli fortemente tipizzati.

Evitare accessi dinamici o dizionari generici quando non necessari.

7. Validazione obbligatoria

Ogni percorso proveniente dalla configurazione deve essere validato.

Prima dell'avvio di Robocopy verificare:

- sorgente esistente;
- destinazione valida;
- sorgente diversa dalla destinazione;
- destinazione non contenuta nella sorgente;
- sorgente non contenuta nella destinazione;
- assenza di caratteri non validi;
- assenza di argomenti shell incorporati;
- disponibilità del disco destinazione;
- cartella log creabile;
- modalità AppData valida;
- Robocopy disponibile.

Non costruire la riga di comando tramite concatenazioni non protette.

Usare `ProcessStartInfo.ArgumentList` quando disponibile.

Quando `ArgumentList` non è utilizzabile, applicare escaping centralizzato e testato.

8. Processo Robocopy

Robocopy deve essere avviato tramite:

```
System.Diagnostics.Process
```

Impostazioni richieste:

```
UseShellExecute = False  
RedirectStandardOutput = True  
RedirectStandardError = True  
CreateNoWindow = True
```

L'output deve essere letto in modo asincrono.

Non usare `WaitForExit()` sul thread UI senza gestione asincrona.

Non bloccare il form.

Non avviare più processi Robocopy contemporaneamente dalla stessa istanza dell'applicazione.

Mantenere il riferimento esatto al processo avviato, necessario per l'interruzione manuale.

9. Comando Robocopy

Il comando base previsto è:

```
/E
/XO
/FFT
/COPY:DAT
/DCOPY:T
/R:2
/W:2
/XJ
/Z
/TEE
/NP
```

La simulazione deve aggiungere:

```
/L
```

Le esclusioni directory devono essere aggiunte tramite:

```
/XD
```

Le esclusioni file devono essere aggiunte tramite:

```
/XF
```

Non modificare i parametri senza motivazione verificata e approvazione.

Ogni parametro deve essere documentato nel codice o in una funzione centralizzata.

10. Regole hardcoded

Le esclusioni critiche devono essere centralizzate in un'unica classe o modulo.

Non duplicare elenchi in più file.

Elenco iniziale:

```
C:\Windows
C:\Program Files
C:\Program Files (x86)
C:\ProgramData
C:\Recovery
C:\PerfLogs
C:\$Recycle.Bin
C:\System Volume Information
```

File inizialmente esclusi:

```
pagefile.sys
hiberfil.sys
swapfile.sys
DumpStack.log
DumpStack.log.tmp
```

Qualunque modifica a questi elenchi richiede approvazione.

11. Architettura minima

Preferire una struttura semplice.

Struttura indicativa:

```
ManualBackup/
├─ ManualBackup.vbproj
├─ FormMain.vb
├─ FormMain.Designer.vb
├─ BackupConfig.vb
├─ BackupRunner.vb
├─ RobocopyCommandBuilder.vb
├─ RobocopyResult.vb
├─ PathValidator.vb
├─ HardcodedExclusions.vb
├─ config.json
├─ CONTRACT.md
├─ AGENTS.md
└─ README.md
```

Questa struttura è indicativa.

Non creare nuovi layer, pattern o progetti separati senza necessità concreta.

Non introdurre:

- repository pattern;
- dependency injection container;
- mediator;
- event bus;
- database;
- ORM;
- microservizi;
- API web;
- servizi background.

12. UI Windows Forms

La UI deve restare semplice.

Non cambiare naming, layout o struttura dei controlli senza approvazione.

Non inserire librerie UI esterne.

Non modificare manualmente `FormMain.Designer.vb` quando la stessa modifica può essere effettuata tramite il designer di Visual Studio.

Quando viene modificato il Designer:

- verificare che il form si apra;
- verificare che il designer non mostri errori;
- verificare gli eventi collegati;
- verificare i nomi dei controlli;
- verificare tab order e accessibilità minima.

13. Thread e asincronia

Le operazioni lunghe non devono essere eseguite sul thread UI.

Usare `Async` e `Await`.

Gli aggiornamenti dei controlli devono avvenire sul thread UI.

Non usare `Application.DoEvents()` come soluzione per mantenere responsiva l'interfaccia.

Non usare thread manuali se `Task` e API asincrone sono sufficienti.

Gestire correttamente:

- avvio;
- completamento;
- eccezione;
- annullamento;
- chiusura del form durante il backup.

14. Interruzione del processo

L'interruzione deve agire solo sul processo Robocopy avviato dall'applicazione.

Non usare comandi generici come:

```
taskkill /IM robocopy.exe
```

Non terminare tutti i processi Robocopy del sistema.

Mantenere l'istanza `Process`.

Prima di interrompere:

- verificare che il processo esista;
- verificare che non sia già terminato;
- chiedere conferma all'utente.

Dopo l'interruzione:

- aggiornare lo stato;
- scrivere il log;
- riabilitare i controlli;
- non mostrare l'operazione come completata.

15. Exit code Robocopy

Implementare una funzione centralizzata per interpretare i codici.

Regola obbligatoria:

```
ExitCode < 8 = completato  
ExitCode >= 8 = errore
```

Non usare:


```
If exitCode = 0 Then
```

come unica condizione di successo.

Il risultato deve distinguere:

```
NoChanges  
Success  
SuccessWithWarnings  
Failure  
FatalFailure  
Cancelled
```

Non cambiare questa semantica senza approvazione.

16. Logging

Ogni esecuzione deve produrre un log.

Non sovrascrivere il log precedente.

Il nome deve includere timestamp e modalità.

Esempio:

```
backup_2026-06-30_153045.log  
simulation_2026-06-30_152210.log
```

Il log deve includere:

- configurazione effettiva non sensibile;
- comando Robocopy;
- output standard;
- output errori;
- exit code;
- risultato interpretato;
- eccezioni applicative;
- eventuale interruzione.

Non scrivere nel log:

- password;
- token;
- credenziali;
- contenuto dei file;
- dati non necessari.

17. Gestione eccezioni

Non usare blocchi `Catch` vuoti.

Ogni eccezione deve essere:

- gestita;
- registrata;
- mostrata in modo comprensibile quando rilevante.

Non mostrare stack trace grezzi all'utente finale.

Lo stack trace può essere inserito nel log tecnico.

Non usare eccezioni come normale flusso applicativo.

18. Dipendenze

Non aggiungere pacchetti NuGet senza approvazione.

Preferire le API incluse in .NET 8.

Per JSON usare:

```
System.Text.Json
```

Per processi usare:

```
System.Diagnostics
```

Per filesystem usare:

```
System.IO
```

Per operazioni asincrone usare:

```
System.Threading.Tasks
```

Non aggiungere librerie per funzioni già disponibili nel framework.

19. Compatibilità

Target obbligatorio iniziale:

```
Windows 11
.NET 8
VB.NET
Windows Forms
```

Non cambiare target framework.

Non convertire il progetto in C#.

Non convertire Windows Forms in WPF, MAUI, Electron o web app.

Non aggiungere compatibilità con altri sistemi operativi senza approvazione.

20. Patch e modifiche

Ogni modifica proposta deve essere completa e verificabile.

Quando viene richiesto un intervento:

- indicare i file modificati;
- fornire la patch completa;
- non omettere sezioni con placeholder;
- non usare frasi come “resto invariato” dentro una patch che deve essere applicabile;
- non rinominare elementi non coinvolti;
- non fare refactoring accessorio;
- non riformattare interi file senza necessità;
- mantenere il diff limitato al problema.

Non includere modifiche estranee nello stesso intervento.

21. Test dopo ogni modifica

Dopo ogni patch verificare almeno:

```
dotnet restore
dotnet build
```

Quando possibile eseguire anche test manuali.

Test minimi per il motore backup:

- simulazione;
- backup con un file nuovo;
- backup con un file modificato;
- secondo backup senza cambiamenti;
- file eliminato dalla sorgente ancora presente nella destinazione;
- percorso non valido;
- destinazione non disponibile;
- exit code inferiore a 8;
- exit code uguale o superiore a 8;
- interruzione manuale.

Non dichiarare una modifica completata se il progetto non compila.

Se non è possibile eseguire un test, dichiararlo esplicitamente.

22. File generati e repository

Non committare:

```
bin/  
obj/  
.vs/  
Logs/  
*.user  
*.suo
```

Non committare log contenenti percorsi personali se il repository è pubblico.

Il file `config.json` può contenere percorsi personali.

Prima di pubblicare il repository valutare l'uso di:

```
config.example.json
```

La decisione sulla gestione di `config.json` deve essere concordata.

23. Sicurezza dei percorsi

Tutti i percorsi devono essere normalizzati tramite API .NET.

Usare:

```
Path.GetFullPath
```

Confrontare i percorsi normalizzati usando una comparazione compatibile con Windows.

Non usare semplici confronti testuali non normalizzati.

Gestire correttamente:

- slash finali;
- maiuscole e minuscole;
- unità differenti;
- percorsi UNC, solo se in futuro approvati;
- spazi;
- caratteri speciali;
- variabili ambiente.

La prima versione non deve espandere comandi o variabili shell arbitrari.

24. Comportamento in caso di dubbio

Quando il contratto non specifica un comportamento:

- non inventare;
- non scegliere autonomamente una nuova architettura;
- non applicare una modifica distruttiva;
- non cambiare contratti esistenti;
- proporre al massimo due opzioni;
- indicare vantaggi e svantaggi;
- richiedere una decisione prima di modificare il progetto.

In assenza di decisione, mantenere il comportamento esistente più conservativo.

25. Priorità operative

Ordine di priorità:

1. sicurezza dei dati;
2. nessuna cancellazione;
3. rispetto di `CONTRACT.md`;
4. correttezza del comando Robocopy;
5. validazione dei percorsi;
6. log affidabile;
7. interfaccia responsiva;
8. semplicità del codice;
9. prestazioni;
10. miglioramenti estetici.

Nessuna ottimizzazione deve compromettere la sicurezza.

26. Definizione di completato

Una modifica è completata solo quando:

- rispetta `CONTRACT.md` ;
- rispetta `AGENTS.md` ;
- compila;
- non introduce cancellazioni;
- non modifica il JSON senza approvazione;
- non aggiunge dipendenze non approvate;
- mantiene la UI responsiva;
- gestisce gli errori;
- produce log;
- è stata verificata con test pertinenti;
- il risultato è documentato.

Questo file deve essere letto prima di ogni intervento sul progetto.